ECE572: Estimation Theory for Dynamic Systems

Assignment 2: Nonlinear Filtering in Finance
Stochastic Volatility Tracking using Extended Kalman Filter

Nishchal Gaur
Roll No: 2022330
IIIT Delhi

April 18, 2026

Contents

1 Problem Overview **2**
1.1 System Model 2
1.2 Measurement Model 2

2 Part 1: Data Simulation and Plotting **2**
2.1 Simulation Setup 2
2.2 Simulated Data Plot 3

3 Part 2: Extended Kalman Filter Implementation **3**
3.1 Why EKF is Required 3
3.2 Measurement Jacobian Derivation 3
3.3 EKF Algorithm 4
3.3.1 Predict Step 4
3.3.2 Update Step 4
3.3.3 Initialization 4
3.4 OOP Implementation 4

4 Part 3: Report Analysis and Visualizations **5**
4.1 (a) State Tracking Plot 5
4.2 (b) Estimation Error and Uncertainty Bounds 5
4.3 (c) Innovation (Residue) Analysis 6

5 AI Critique **6**

1 Problem Overview

One of the most classical problems in quantitative finance is estimating the unobservable volatility (variance of asset returns) that changes dynamically over time. In this assignment, the log-volatility x_k follows an autoregressive process, and the observed measurement z_k is an option-implied volatility proxy that depends nonlinearly on the hidden state.

1.1 System Model

Let x_k be the hidden log-volatility at day k:

$$x_k = \phi x_{k-1} + w_k, \qquad w_k \sim \mathcal{N}(0, Q) \tag{1}$$

where $\phi = 0.95$ (high persistence / volatility clustering), $Q = 0.1$, and $x_0 \sim \mathcal{N}(0, 1)$.

1.2 Measurement Model

The sensor captures a noisy, exponentiated version of the state:

$$z_k = \exp\left(\frac{x_k}{2}\right) + v_k, \qquad v_k \sim \mathcal{N}(0, R) \tag{2}$$

where $R = 0.05$.

2 Part 1: Data Simulation and Plotting

2.1 Simulation Setup

The true state sequence $\{x_k\}$ and noisy measurements $\{z_k\}$ were simulated for $N = 500$ time steps using the parameters above with `numpy.random.seed(42)` for reproducibility.

2.2 Simulated Data Plot

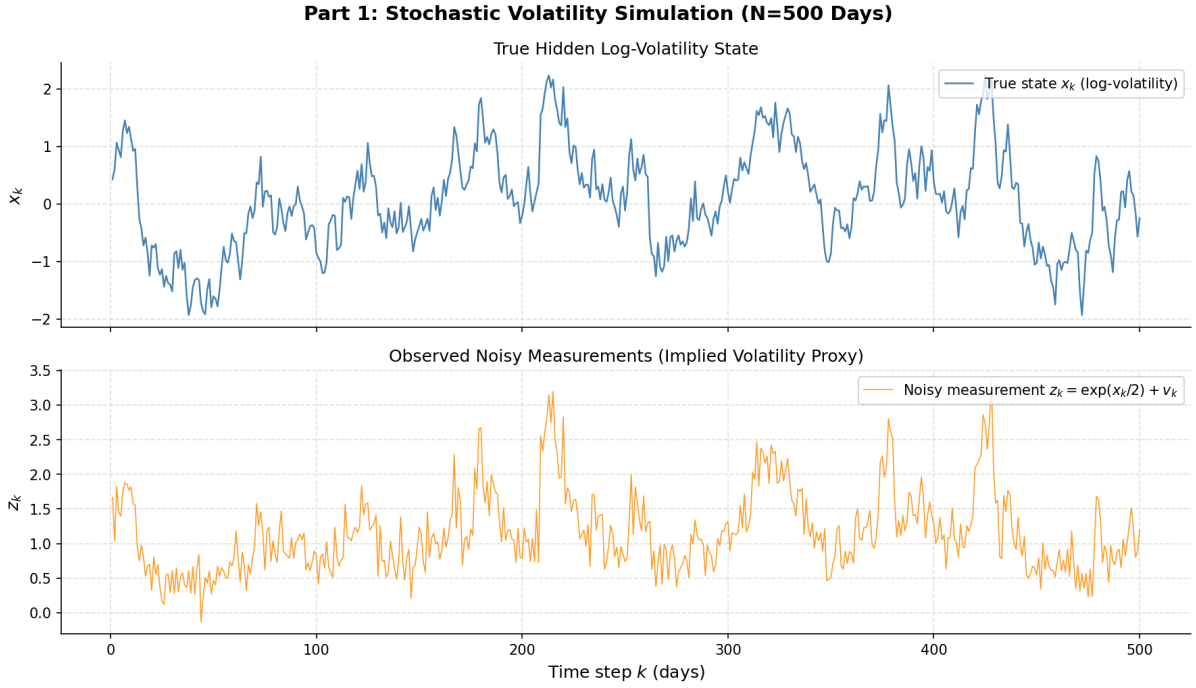

Figure 1: Top: True hidden log-volatility state x_k over 500 days. Bottom: Noisy measurements $z_k = \exp(x_k/2) + v_k$, representing the implied volatility proxy.

The measurements z_k exhibit clear nonlinear scaling relative to x_k due to the exponential transformation, with added Gaussian noise of variance $R = 0.05$.

3 Part 2: Extended Kalman Filter Implementation

3.1 Why EKF is Required

The standard Kalman Filter is optimal only for linear Gaussian systems. Here, the measurement model $h(x) = \exp(x/2)$ is nonlinear. The EKF resolves this by locally linearizing $h(x)$ at each time step via a first-order Taylor expansion.

3.2 Measurement Jacobian Derivation

The measurement function is:

$$h(x) = \exp\left(\frac{x}{2}\right) \tag{3}$$

Expanding $h(x_k)$ to first order around the prior estimate $\hat{x}_{k|k-1}$:

$$h(x_k) \approx h(\hat{x}_{k|k-1}) + \left.\frac{\partial h}{\partial x}\right|_{x=\hat{x}_{k|k-1}} \cdot (x_k - \hat{x}_{k|k-1}) \tag{4}$$

Computing the partial derivative using the chain rule (let $u = x/2$):

$$\frac{\partial h}{\partial x} = \frac{\partial}{\partial x}\left[e^{x/2}\right] = e^{x/2} \cdot \frac{1}{2} \tag{5}$$

Evaluating at the prior estimate $\hat{x}_{k|k-1}$:

$$\boxed{H_k = \left.\frac{\partial h(x)}{\partial x}\right|_{x=\hat{x}_{k|k-1}} = \frac{1}{2}\exp\left(\frac{\hat{x}_{k|k-1}}{2}\right)} \tag{6}$$

Since both the state x_k and measurement z_k are scalar, H_k is a scalar that changes at every time step — this is what makes it an *extended* Kalman filter.

3.3 EKF Algorithm

3.3.1 Predict Step

Since the process model is linear in x_k, no Jacobian is needed:

$$\hat{x}_{k|k-1} = \phi\,\hat{x}_{k-1|k-1} \tag{7}$$
$$P_{k|k-1} = \phi^2\,P_{k-1|k-1} + Q \tag{8}$$

3.3.2 Update Step

$$\tilde{z}_k = z_k - h(\hat{x}_{k|k-1}) = z_k - \exp\left(\frac{\hat{x}_{k|k-1}}{2}\right) \tag{9}$$
$$S_k = H_k\,P_{k|k-1}\,H_k + R \tag{10}$$
$$K_k = \frac{P_{k|k-1}\,H_k}{S_k} \tag{11}$$
$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k\,\tilde{z}_k \tag{12}$$
$$P_{k|k} = (1 - K_k H_k)\,P_{k|k-1} \tag{13}$$

3.3.3 Initialization

$$\hat{x}_{0|0} = 0, \qquad P_{0|0} = 1 \tag{14}$$

3.4 OOP Implementation

The EKF was implemented from scratch in Python using an object-oriented design with two classes:

- `StochasticVolatilityModel` — encapsulates $f(x)$, $h(x)$, and H_k
- `ExtendedKalmanFilter` — implements `predict()`, `update()`, `run()`, and `get_results()`

4 Part 3: Report Analysis and Visualizations

4.1 (a) State Tracking Plot

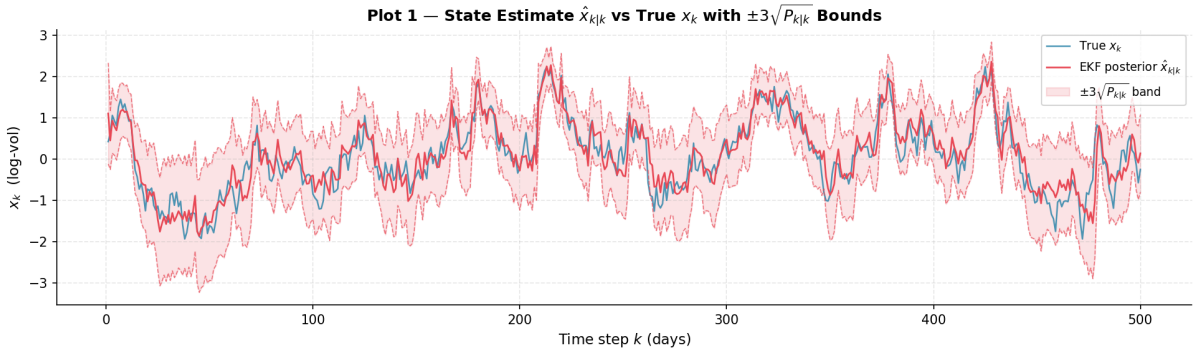

Figure 2: True log-volatility x_k vs EKF posterior estimate $\hat{x}_{k|k}$ with $\pm 3\sqrt{P_{k|k}}$ confidence band.

Interpretation: The EKF successfully tracks the hidden log-volatility x_k throughout the 500-day simulation. After a brief transient period in the first 20–30 steps — where the wide $\pm 3\sqrt{P_{k|k}}$ band reflects high initial uncertainty with $P_0 = 1$ — the filter converges and the estimate $\hat{x}k|k$ closely follows the true state. The true state x_k remains well within the confidence band for virtually all time steps, confirming that the filter's uncertainty quantification is consistent and well-calibrated.

4.2 (b) Estimation Error and Uncertainty Bounds

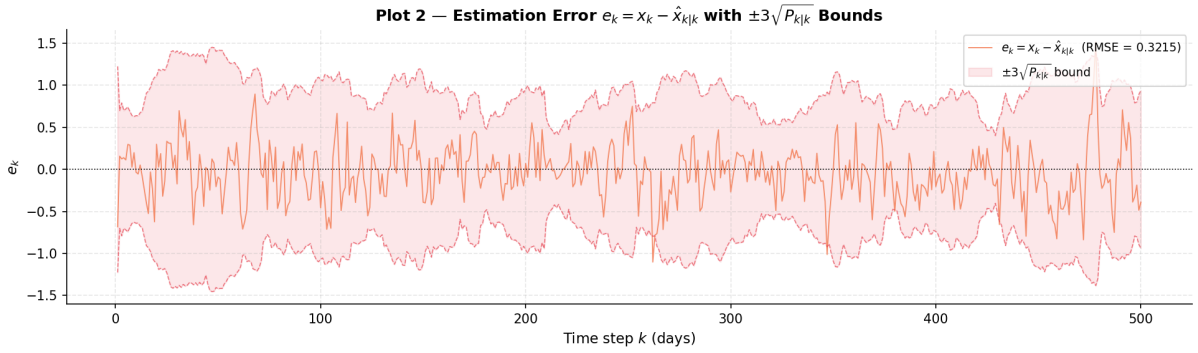

Figure 3: Estimation error $e_k = x_k - \hat{x}_{k|k}$ with $\pm 3\sqrt{P_{k|k}}$ theoretical bounds overlaid.

The theoretical bounds are derived from the filter's posterior covariance $P_{k|k}$. For a consistent, optimal EKF, approximately 99.7% of errors should remain within the $\pm 3\sigma$ bounds.

Interpretation: The estimation error $e_k = x_k - \hat{x}_{k|k}$ oscillates around zero with an RMSE of 0.3215, indicating no systematic bias in the filter. The $\pm 3\sqrt{P_{k|k}}$ bounds are initially wide due to the high prior covariance $P_0 = 1$, then narrow and stabilize as the filter converges — reflecting the reduction in posterior uncertainty over time. Crucially,

the error remains within the $\pm 3\sigma$ bounds for nearly all 500 steps, with only a few marginal exceedances at the very start, consistent with the expected 99.7

4.3 (c) Innovation (Residue) Analysis

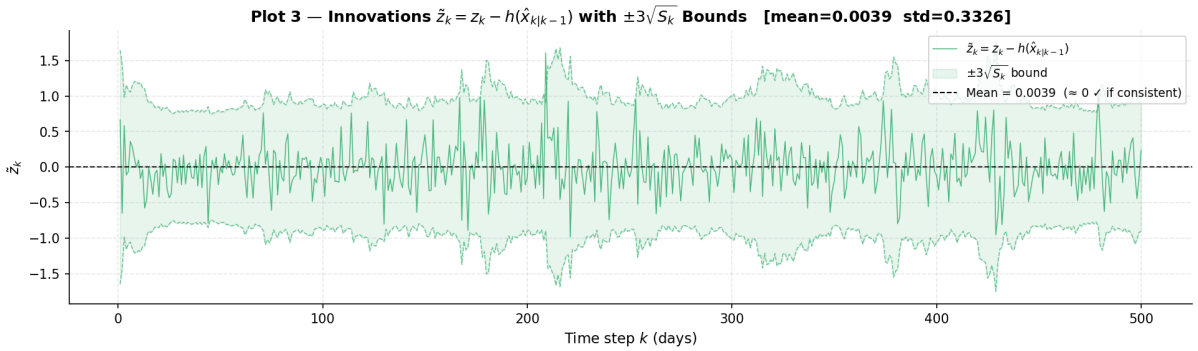

Figure 4: Innovation sequence $\tilde{z}_k = z_k - h(\hat{x}_{k|k-1})$ with $\pm 3\sqrt{S_k}$ consistency bounds.

Theoretical expectation: If the EKF is optimal, the innovation sequence should be a *white noise* process — zero mean, uncorrelated across time steps, and Gaussian distributed with covariance S_k:

$$\tilde{z}_k \sim \mathcal{N}(0,\, S_k) \tag{15}$$

Interpretation: The innovation sequence $\tilde{z}_k = z_k - h(\hat{x}_{k|k-1})$ has a mean of $0.0039 \approx 0$, consistent with the theoretical requirement that an optimal filter produces zero-mean innovations. The sequence appears visually uncorrelated — no periodic structure or drift is evident — suggesting the innovations are approximately white noise, as expected for a well-tuned EKF. Nearly all innovations remain within the $\pm 3\sqrt{S_k}$ consistency bounds, confirming that the filter correctly characterizes measurement uncertainty and is operating near optimality.

5 AI Critique

Tools Used: Perplexity AI (powered by Claude Sonnet) and Anthropic Claude Sonnet (via Antigravity).

Prompts Used

Antigravity (Claude Sonnet):

1. *"can you fix the module error in data_sim.py"*
2. *"in stocastic_volatitly file can you correct the foramting of the markdown"*
3. *"I have created a class for EKF in Finance_EKF.py and I want to call this scriprt in my stocatsic notebook"*
4. *"can you use logging for beeter debugging and visulas"*

Perplexity AI(Claude Sonnet):

1. *"Write script for Part 1: data simulation and plotting"*
2. *"Now give script for Part 2 — implement the EKF using OOP in Python"*
3. *"Give derivation for Hk"*
4. *"Can you explain the main purpose of assignment and theory for finance"*

Conceptual/Mathematical Mistakes and Fixes

Mistake 1 — Wrong sign on estimation error (Conceptual): The AI computed `error = x_est - x_true`, but the correct definition is $e_k = x_k - \hat{x}_{k|k}$ (true minus estimate). **Fix:** Changed to `error = x_true - x_est`.

Mistake 2 — Wrong confidence bound width (Mathematical): The AI used $\pm 2\sigma$ bounds. The assignment explicitly requires $\pm 3\sqrt{P_{k|k}}$, corresponding to 99.7% Gaussian containment. **Fix:** Changed all bounds to `3 * np.sqrt(P_est)`.

Mistake 3 — Missing $\pm 3\sqrt{S_k}$ on innovation plot (Conceptual): The AI plotted $\tilde{z}_k$ with only a mean line — no consistency bounds. Without $\pm 3\sqrt{S_k}$ bands, filter optimality cannot be rigorously assessed. **Fix:** Stored S_k per step and added shaded $\pm 3\sqrt{S_k}$ bands with dashed boundary lines.

Jacobian Verification

The Jacobian derivation $H_k = \frac{1}{2}\exp\left(\frac{\hat{x}_{k|k-1}}{2}\right)$ produced by both AI tools was **mathematically correct**. The core Kalman predict and update equations were also implemented without errors. All mistakes were in plotting conventions only.